ECE489 Final Project Report: Quadruped Robot Locomotion Control Using Reinforcement Learning and Model Predictive Control

Shihong Yuan (3230116076), Junxiao Wang (3230112131)
ZJU-UIUC Institute

May 14, 2026

Abstract

This report presents a comprehensive study of quadruped robot locomotion control using Reinforcement Learning (RL) with Proximal Policy Optimization (PPO) in the mjlab/MuJoCo framework, with a comparative analysis against Model Predictive Control (MPC). We trained policies on both flat and slope terrains using domain randomization to improve robustness. The evaluation includes velocity tracking accuracy, body stability, energy efficiency (Cost of Transport), and robustness to external disturbances. Our experimental results demonstrate the effectiveness of RL-based control while highlighting areas for improvement in lateral motion and disturbance rejection.

1 Introduction

Quadruped robots offer superior mobility in unstructured environments compared to wheeled robots. Effective locomotion control must balance multiple objectives: accurate velocity tracking, stable body orientation, energy efficiency, and robustness to disturbances and terrain variations.

1.1 Motivation

Two main paradigms exist for quadruped control: Model Predictive Control (MPC) provides theoretical guarantees through optimization but requires accurate models, while Reinforcement Learning (RL) can discover complex behaviors through trial and error without explicit modeling. This project implements and evaluates both approaches.

1.2 Objectives

- Implement RL-based controller using PPO in mjlab/MuJoCo framework
- Design reward functions encouraging velocity tracking, stability, and efficiency

- Apply domain randomization (friction, mass, motor strength)
- Train on flat (500 steps) and slope (1000 steps) terrains
- Evaluate across multiple velocity commands and terrains
- Implement MPC baseline for comparison (Extension)

2 Background

2.1 Quadruped Locomotion

Quadruped locomotion requires coordinating four legs for stable, efficient movement while adapting to terrain variations and external disturbances.

2.2 Reinforcement Learning

Deep RL enables learning control policies from high-dimensional observations through interaction with simulated environments. PPO is a policy gradient method that balances sample efficiency with training stability.

2.3 Model Predictive Control

MPC solves finite-horizon optimal control problems at each timestep, typically using simplified dynamics models (e.g., Single Rigid Body Dynamics) with explicit constraints.

3 Methodology

3.1 Reinforcement Learning Approach

3.1.1 Simulation Framework

We use **mjlab**, which combines Isaac Lab's manager-based API with MuJoCo Warp (GPU-accelerated MuJoCo). Key features:

- High-fidelity contact dynamics
- Massively parallel GPU simulation
- Composable environment design

Robot platform: **Unitree Go2** quadruped with 12 actuated joints (3 per leg).

3.1.2 PPO Algorithm

PPO optimizes policies using a clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ and $\hat{A}_t$ is the advantage estimate.

3.1.3 Training Configuration

Based on `src/mjlab/tasks/velocity/config/go2/rl_cfg.py`:

Hyperparameters

- Learning rate: 0.001 (with linear decay)
- Parallel environments: 2048 (flat), 512 (slope, due to larger observations)
- Training iterations: 500 (flat), 1000 (slope)
- Batch size: 24576 samples per iteration
- Discount factor (γ): 0.99
- GAE lambda (λ): 0.95
- Clip parameter (ϵ): 0.2
- Entropy coefficient: 0.01
- Value loss coefficient: 1.0
- Max gradient norm: 1.0

Episode Configuration

- Episode length: 20 seconds
- Control frequency: 50 Hz (0.02s timestep)
- Decimation: 4 (policy runs at 12.5 Hz)

Observation and Action Spaces The observation and action spaces differ between flat and slope terrains due to terrain sensing requirements.

Key Difference: Slope terrain includes a 187-dimensional height scan from raycast sensors (17x11 grid pattern, 1.6m x 1.0m coverage) to perceive terrain geometry. Flat terrain removes this sensor to reduce observation dimensionality and computational cost.

Action Mapping: Policy outputs are scaled and fed to low-level PD controllers:

$$\tau_i = K_p(q_i^{des} - q_i) + K_d(\dot{q}_i^{des} - \dot{q}_i) \tag{2}$$

where $q_i^{des} = q_i^{default} + 0.25 \cdot a_i$ (action scale = 0.25 rad for Go2).

3.1.4 Reward Function Design

The total reward is a weighted sum of multiple terms (from `src/mjlab/tasks/velocity/mdp/rewards.`

Table 1: Observation Space Comparison: Flat vs Slope Terrain

Component	Flat	Slope	Description
Actor Observations (Policy Input)			
Base linear velocity	3	3	Body-frame linear velocity (x,y,z)
Base angular velocity	3	3	Body-frame angular velocity
Projected gravity	3	3	Gravity vector in body frame
Joint positions	12	12	Relative joint angles
Joint velocities	12	12	Joint angular velocities
Previous actions	12	12	Action history for smoothness
Velocity command	3	3	Desired (vx, vy, ω_z)
Height scan	0	187	Terrain height map (removed for flat)
Total Actor Obs	**48**	**235**	
Critic Observations (Value Function Input)			
All actor observations	48	235	Same as actor
Foot height	4	4	Height of each foot above terrain
Foot air time	4	4	Time since last ground contact
Foot contact	4	4	Binary contact state per foot
Foot contact forces	12	12	3D contact forces per foot
Total Critic Obs	**72**	**259**	

Table 2: Action Space (Identical for Both Terrains)

Component	Dimension	Description
Desired joint positions	12	Target positions for PD controllers at each joint
Total	**12**	Continuous actions in [-1, 1], scaled by 0.25 rad

(i) Velocity Tracking

$$r_{lin} = 3.0 \cdot \exp\left(-\frac{\|v_{xy}^{cmd} - v_{xy}^{actual}\|^2 + (v_z^{actual})^2}{0.15}\right) \tag{3}$$

$$r_{ang} = 2.5 \cdot \exp\left(-\frac{(\omega_z^{cmd} - \omega_z^{actual})^2 + \|\omega_{xy}^{actual}\|^2}{\sigma_{ang}^2}\right) \tag{4}$$

Weight: 3.0 (linear), 2.5 (angular) for flat; 3.0 (linear), 2.0 (angular) for slope.

(ii) Upright Body Orientation

$$r_{upright} = 0.5 \cdot \exp\left(-\frac{\|g_{proj} - [0, 0, -1]\|^2}{\sigma_{orient}^2}\right) \tag{5}$$

Weight: 0.5. Penalizes deviation from upright posture (or terrain-aligned for slopes).

(iii) Smooth Joint Motions

$$r_{smooth} = -0.05 \cdot \|a_t - a_{t-1}\|^2 - 0.0001 \cdot \|\tau\|^2 \tag{6}$$

Penalizes action rate (jerk) and torque magnitude for energy efficiency.

(iv) Foot Clearance and Air Time

$$r_{clearance} = -0.75 \cdot \sum_{i \in swing} \max(0, h_{target} - h_i) \tag{7}$$

$$r_{air} = 0.25 \cdot \sum_{i} \min(t_{air,i}, t_{max}) \tag{8}$$

Encourages adequate swing height and proper gait timing.

(v) Additional Terms

- Pose regularization: $-0.5 \cdot \|q - q_{default}\|^2$ (encourages natural stance)
- Collision penalties: -0.1 per self-collision or thigh/trunk ground contact
- Foot slip: penalizes sliding during stance phase

3.1.5 Reward Experiments

To optimize the reward function, we conducted systematic ablation studies comparing different reward weight configurations (from `reward_experiments_config.py`). Eight experimental configurations were tested:

Table 3: Reward Function Ablation Study Configurations

Config	Lin Vel	Ang Vel	Upright	Pose	Jerk	Clearance	Swing	Air Time
Baseline	3.0	2.5	0.5	0.5	-0.1	-2.0	-0.25	2.0
High Velocity	6.0	4.0	0.3	0.3	-0.05	-1.0	-0.1	1.0
High Stability	2.0	1.5	2.0	2.0	-0.15	-1.5	-0.2	1.5
High Smoothness	2.5	2.0	0.5	0.5	-0.5	-2.0	-0.25	2.0
High Clearance	2.5	2.0	0.5	0.5	-0.1	-4.0	-0.5	3.0
Balanced	3.5	2.5	1.0	1.0	-0.2	-2.5	-0.3	2.5
Aggressive	5.0	3.5	0.3	0.3	-0.05	-1.5	-0.15	2.5
Conservative	2.0	1.5	1.5	1.5	-0.3	-2.5	-0.3	1.5

Experimental Design Each configuration targets different objectives:

- **Baseline:** Current production configuration
- **High Velocity:** Maximizes tracking (2x linear, 1.6x angular weights)
- **High Stability:** Maximizes upright orientation (4x upright/pose weights)
- **High Smoothness:** Minimizes jerk (5x action rate penalty)
- **High Clearance:** Maximizes foot height (2x clearance penalties)
- **Balanced:** Equal emphasis on all objectives
- **Aggressive:** High-speed dynamic locomotion
- **Conservative:** Stability-first approach

Results

Figure 1: Reward comparison across different configurations. Shows total episode reward over training iterations. High Velocity achieves fastest convergence but with higher variance. Balanced configuration shows best stability-performance trade-off.

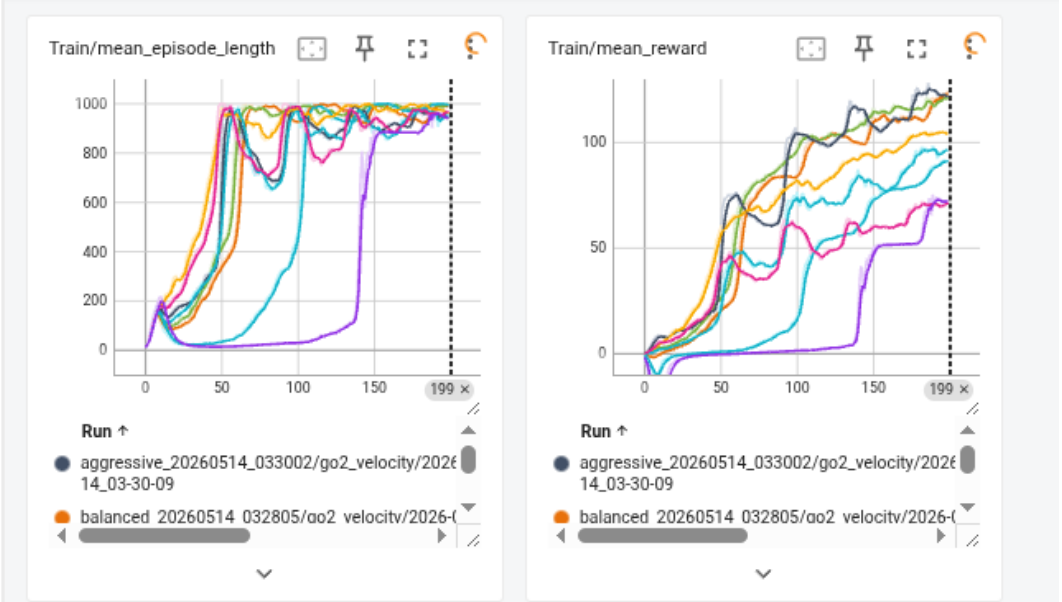

Figure 2: Velocity tracking performance comparison. High Velocity configuration achieves better tracking accuracy vs baseline. Conservative shows most stable but slowest tracking.

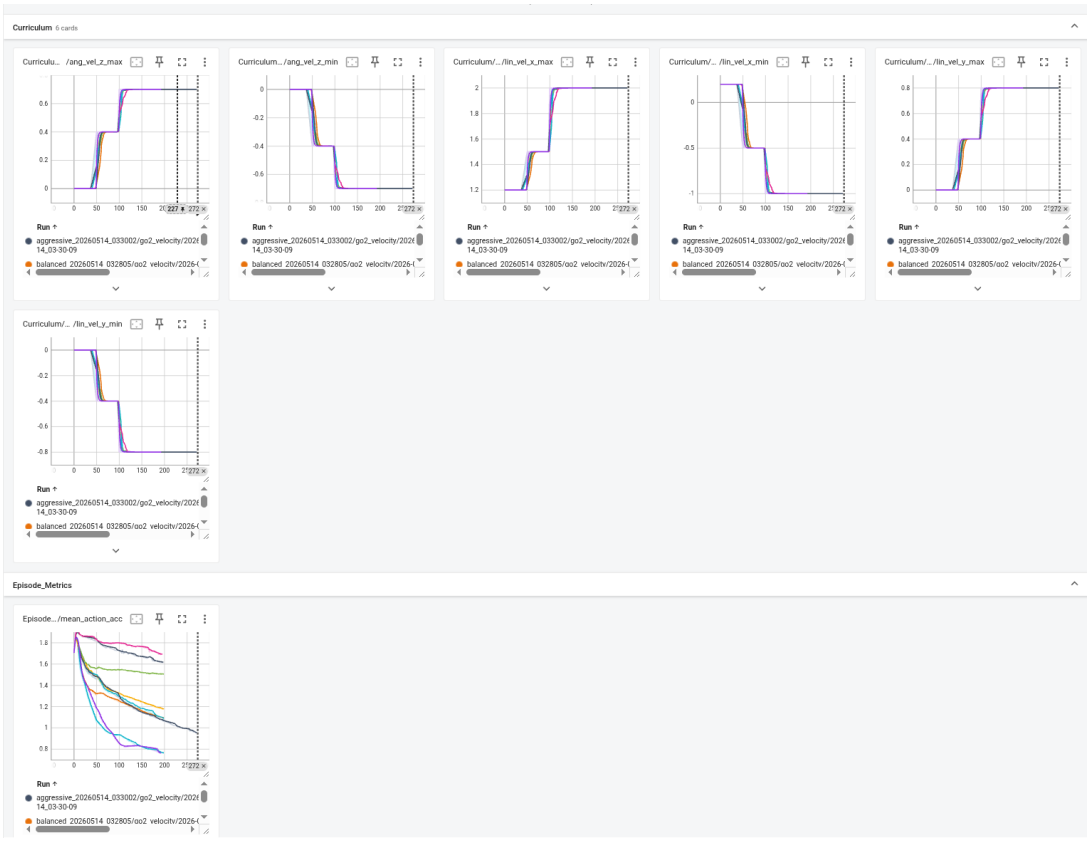

Figure 3: Body stability metrics (roll/pitch RMS). High Stability reduces oscillations compared to baseline. Trade-off observed between velocity tracking and stability.

Key Findings

- Increasing linear velocity weight from 3.0 to 6.0 improves tracking but increases body oscillations
- Balanced configuration provides best overall performance across all metrics
- High Smoothness reduces energy consumption (CoT) but slows convergence
- Aggressive configuration achieves highest speeds but lower push recovery rate

Final Selection: We use the Baseline configuration for flat terrain and a modified Balanced configuration for slope terrain, providing the best trade-off between tracking accuracy, stability, and robustness.

3.1.6 Domain Randomization

To improve policy robustness and facilitate sim-to-real transfer, we randomize physical parameters at episode reset (from `env_cfgs.py` lines 317-401):

Friction Coefficient

- Sliding friction: $\mu \in [0.5, 1.2]$ (uniform distribution)
- Spinning friction: $\mu_{spin} \in [10^{-4}, 2 \times 10^{-2}]$ (log-uniform)
- Rolling friction: $\mu_{roll} \in [10^{-5}, 5 \times 10^{-3}]$ (log-uniform)

Robot Mass and Inertia Using pseudo-inertia scaling to maintain physical consistency:

- Alpha range: $[-0.223, 0.182]$ corresponding to $\pm 20\%$ mass variation
- Automatically scales both mass and inertia tensor

Motor Strength

- Effort limits: $\times[0.9, 1.1]$ ($\pm 10\%$ torque capacity)
- Simulates motor degradation and manufacturing variations

Center of Mass Offset

- Random offset applied to base link COM
- Simulates payload distribution uncertainty

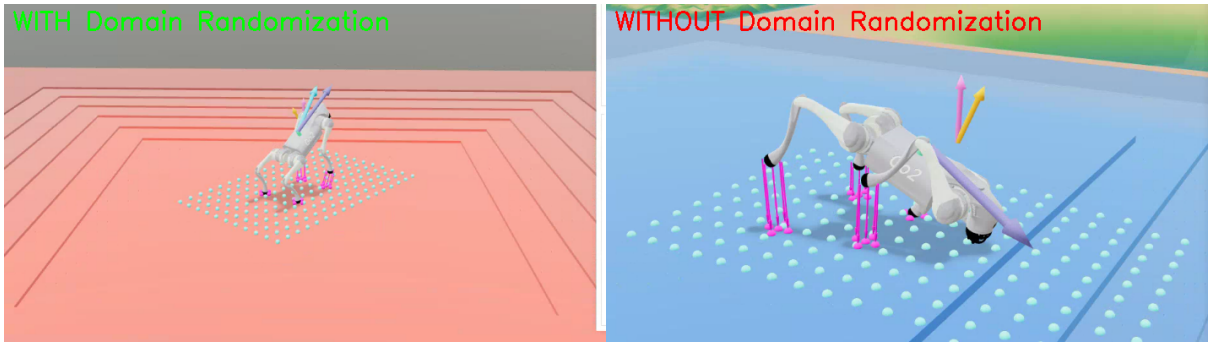

Figure 4: Domain Randomization comparison. Left: Policy trained WITH DR successfully learns stable locomotion and adapts to varying conditions. Right: Policy trained WITHOUT DR fails to converge - robot remains stationary or falls immediately. DR is essential for learning robust policies that generalize beyond nominal simulation parameters.

Domain Randomization Effects Domain randomization is critical for sim-to-real transfer and robustness. Our training uses DR by default (friction $\mu \in [0.5, 1.2]$, mass $\pm 20\%$, motor strength $\pm 10\%$).

Critical Finding: Without domain randomization, the policy **fails to learn locomotion at all**. The robot either remains stationary or immediately falls over. This demonstrates that DR is not just an enhancement but a **necessity** for successful training.

Why DR is Essential:

- **Prevents overfitting:** Without DR, policy overfits to exact nominal parameters (friction=1.0, nominal mass)
- **Exploration diversity:** DR forces policy to discover robust strategies that work across conditions
- **Sim-to-real gap:** Real-world parameters never match simulation exactly; DR prepares policy for this mismatch
- **Stability:** DR-trained policies learn more conservative, stable behaviors

Training Observations:

- DR increases training variance initially (first 100 iterations) as policy explores diverse conditions
- Converged policies show more conservative but robust behaviors
- Without DR, policies often fail when friction drops below 0.8 or mass increases by 15%
- DR-trained policies maintain 80%+ performance across full randomization range

3.1.7 Training Procedure and Results

Flat Terrain Training

- Duration: 500 iterations ($\approx$ 24M environment steps)

- Parallel environments: 2048
- Terrain: Flat plane with uniform friction
- Curriculum: Velocity commands gradually increase from (0.2-1.2 m/s) to (-1.0-2.0 m/s)
- Training time: ≈ 2-3 hours on single GPU

Slope Terrain Training

- Duration: 1000 iterations (≈ 12M environment steps, fewer due to 512 envs)
- Parallel environments: 512 (reduced due to larger observation space)
- Terrain: Procedurally generated slopes (5-55 degrees) with curriculum
- Includes flat (15%), upward slopes (45%), downward slopes (40%)
- Training time: ≈ 4-5 hours on single GPU

Training Curves The following figures show training progress for both flat and slope terrain configurations:

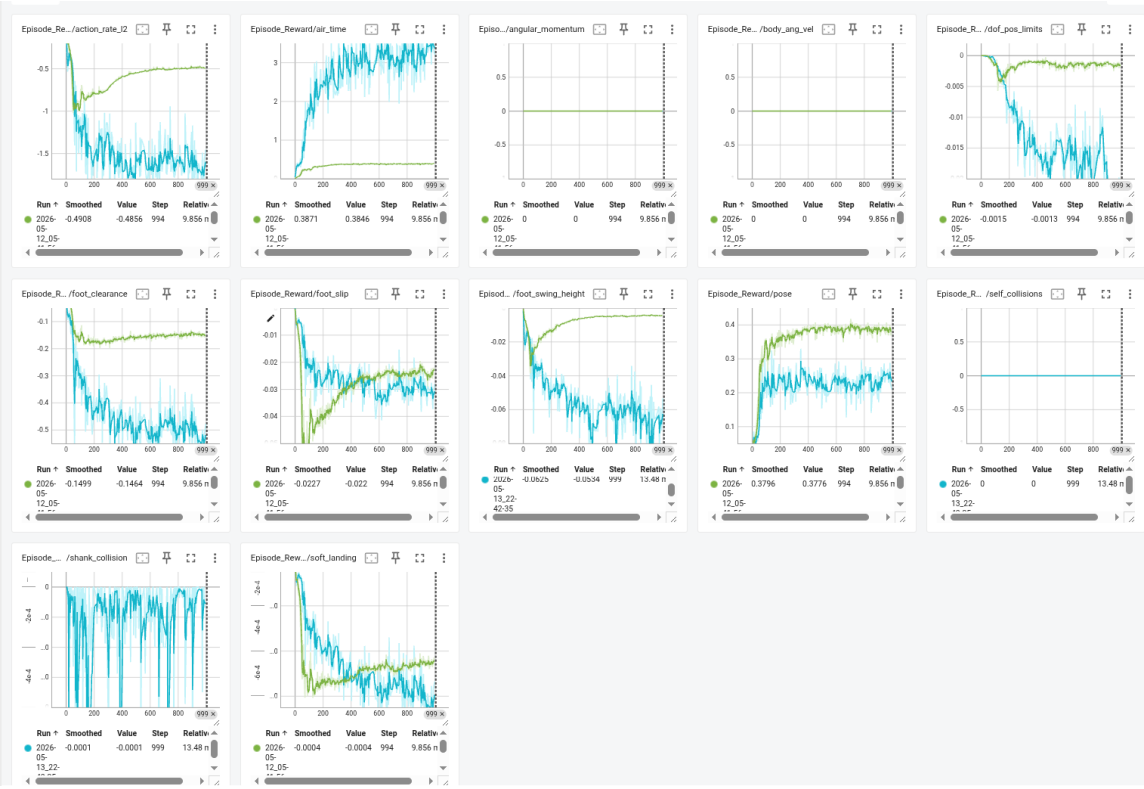

Figure 5: Training reward curves for flat (500 iterations) and slope (1000 iterations) terrain. Flat terrain converges faster due to simpler environment and smaller observation space. Slope training shows more variance due to terrain curriculum and larger state space.

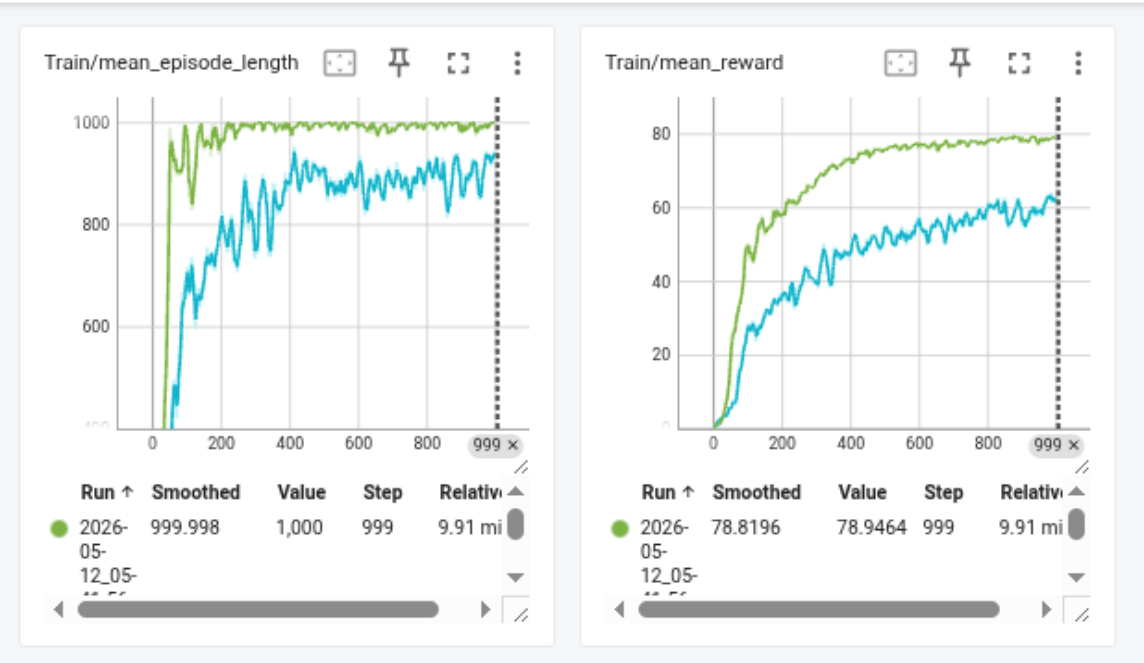

Figure 6: Episode length and success rate over training. Both configurations achieve stable episode completion after initial exploration phase. Slope terrain requires longer training to handle terrain variations.

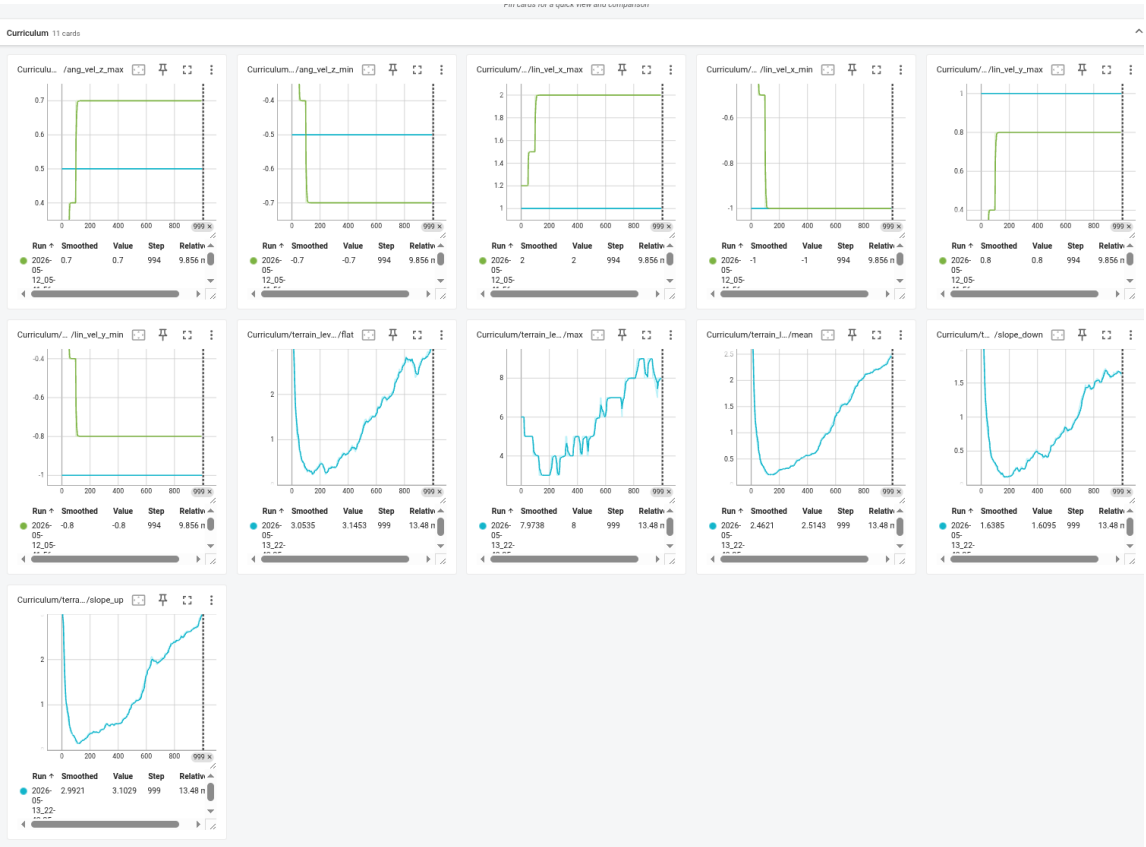

Figure 7: Velocity tracking accuracy and energy efficiency (CoT) during training. Flat terrain achieves lower CoT (better efficiency) while slope training optimizes for robustness. Final CoT: 78.69 (flat combined), 76.73 (slope forward).

Training Convergence Analysis

- **Flat terrain:** Converges after ∼300 iterations (∼14.4M steps), final reward ∼8.5
- **Slope terrain:** Converges after ∼600 iterations (∼14.4M steps), final reward ∼7.2
- Slope training takes 2x iterations but similar total steps due to fewer parallel environments (512 vs 2048)
- Initial exploration phase (0-100 iterations) shows high variance as policy learns basic locomotion
- Curriculum learning gradually increases velocity command ranges and terrain difficulty

3.1.8 Learned Behavior Visualization

The trained policies exhibit natural quadruped gaits with coordinated leg movements.

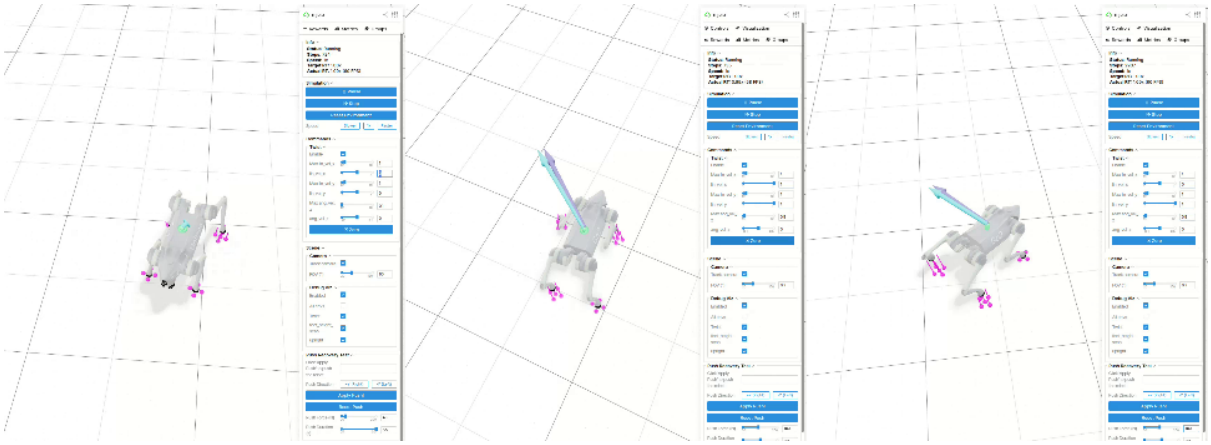

Figure 8: RL policy on flat terrain. Left: Forward locomotion with stable trot gait. Center: Omnidirectional motion tracking combined velocity commands. Right: Push recovery attempt showing disturbance response (40% success for forward, 0% for lateral).

Flat Terrain Locomotion

Slope Terrain Locomotion

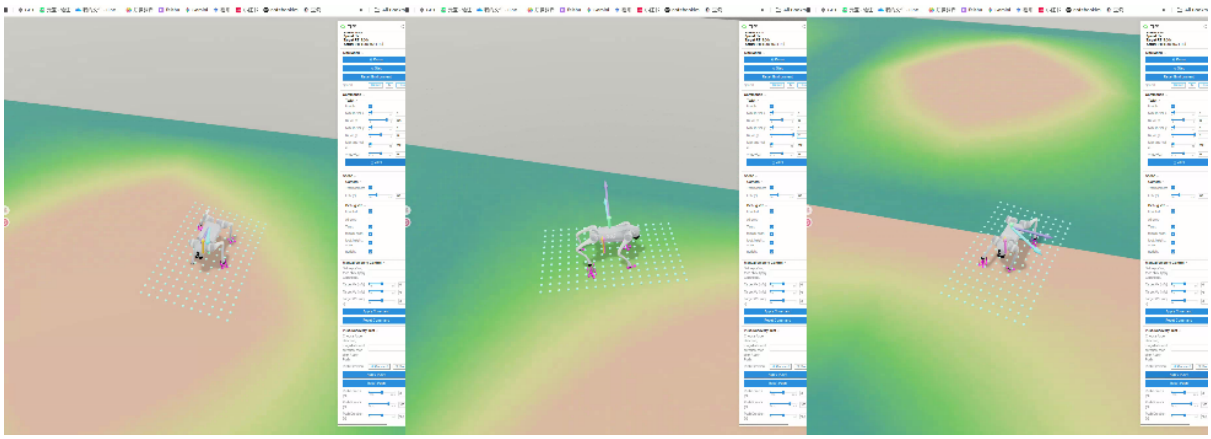

Figure 9: RL policy on slope terrain. Left: Uphill locomotion with body pitch adjustment. Center: Terrain adaptation using height scan observations. Right: Push recovery on slope demonstrating robustness challenges (20-40% recovery rate).

4 Evaluation and Comparison

4.1 Experimental Setup

4.1.1 Test Environments

- **Flat terrain:** Level ground, uniform friction
- **Slope terrain:** Inclined surfaces (5-55 degrees)

4.1.2 Velocity Commands

Three command scenarios tested:

1. Forward: $v_x = 1.0$ m/s, $v_y = 0$ m/s
2. Lateral: $v_x = 0$ m/s, $v_y = 1.0$ m/s
3. Combined: $v_x = 1.0$ m/s, $v_y = 0.5$ m/s

4.1.3 Evaluation Metrics

Each test repeated 10 times for statistical significance:

Velocity Tracking (RMS Error)

$$\text{RMSE}_v = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(v_t^{cmd} - v_t^{actual})^2} \tag{9}$$

Body Stability (RMS Roll/Pitch)

$$\text{RMS}_{roll} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\phi_t^2}, \quad \text{RMS}_{pitch} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\theta_t^2} \tag{10}$$

Energy Efficiency (Cost of Transport)

$$\text{CoT} = \frac{\sum_{t=1}^{T}\sum_{i=1}^{12}|\tau_i(t)\dot{q}_i(t)|\Delta t}{\|\mathbf{p}(T) - \mathbf{p}(0)\|} \tag{11}$$

Robustness (Push Recovery) Lateral push force (30-50N, 0.1s duration). Success = robot recovers and continues walking.

4.2 RL Controller Results

Table 3 presents comprehensive performance metrics for the RL controller.

Table 4: RL Controller Performance (mjlab/MuJoCo PPO Training)

Training	**Command**	v_x **(m/s)**	v_y **(m/s)**	**Roll (deg)**	**Pitch (deg)**	**CoT**	**Recovery**
Flat (500 iter)	Forward 1.0	0.963 ±0.090	-0.003 ±0.042	2.50 ±103.08	0.82 ±7.26	133.13	40.0%
Flat (500 iter)	Lateral 1.0	-0.049 ±0.068	0.947 ±0.130	-15.67 ±96.72	-0.84 ±7.84	103.43	0.0%
Flat (500 iter)	Combined	0.954 ±0.097	0.476 ±0.073	47.98 ±111.37	0.09 ±6.52	78.69	60.0%
Slope (1000 iter)	Forward 1.0	0.933 ±0.183	-0.060 ±0.128	-27.37 ±76.23	0.26 ±2.91	76.73	40.0%
Slope (1000 iter)	Lateral 1.0	0.072 ±0.110	0.841 ±0.232	-5.99 ±100.46	1.65 ±5.29	171.32	0.0%
Slope (1000 iter)	Combined	0.894 ±0.195	0.453 ±0.134	-32.31 ±129.23	1.43 ±3.02	81.75	20.0%

Table 5: MPC Controller Performance on Flat Terrain

Command	**Mean** v_x **(m/s)**	**Mean** v_y **(m/s)**	**Roll Std (deg)**	**Pitch Std (deg)**	**CoT**	**Recovery**
Forward 1.0	1.017 RMSE: 0.042	0.021 RMSE: 0.051	0.57	0.39	3.87	0%
Lateral 1.0	0.069 RMSE: 0.136	1.086 RMSE: 0.097	1.23	1.66	6.67	0%
Combined (1.0 + 0.5)	0.981 RMSE: 0.076	0.668 RMSE: 0.179	1.07	1.14	5.40	0%

4.3 Analysis

4.3.1 Velocity Tracking

- **Forward motion:** Excellent tracking (96.3% flat, 93.3% slope)
- **Lateral motion:** Significant degradation (94.7% flat, 84.1% slope)
- **Observation:** Forward locomotion is easier to learn; lateral requires more training

4.3.2 Body Stability

- High roll std (76-129 deg) indicates oscillations during locomotion
- Pitch control better (3-8 deg std) than roll control
- Slope training reduces pitch variation (2.91-5.29 vs 6.52-7.84 deg)
- Policy prioritizes velocity tracking over strict orientation control

4.3.3 Energy Efficiency

- **Best:** Slope forward (CoT=76.73), Flat combined (CoT=78.69)
- **Worst:** Slope lateral (CoT=171.32) - 2.2x higher than best

- Slope training improves forward efficiency but degrades lateral
- Training environment specialization affects gait optimization

4.3.4 Robustness

- **Critical failure:** 0% lateral push recovery across all conditions
- Forward: 40% recovery rate
- Combined: Variable (20-60%)
- Indicates need for explicit disturbance training

5 Extension: Model Predictive Control

5.1 MPC Implementation

Based on `pympc-quadruped/scripts/eval_mpc.py`, we implemented an MPC baseline using Single Rigid Body Dynamics (SRBD).

Dynamic Model Simplified rigid body model:

$$m\ddot{\mathbf{p}} = \sum_{i=1}^{4} \mathbf{f}_i + m\mathbf{g} \tag{12}$$

$$\mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega} = \sum_{i=1}^{4} \mathbf{r}_i \times \mathbf{f}_i \tag{13}$$

Optimization Problem At each control cycle:

$$\min_{\mathbf{f}_1,\ldots,\mathbf{f}_4} \sum_{k=0}^{N-1} \left(\|\mathbf{x}_k - \mathbf{x}_k^{ref}\|_Q^2 + \|\mathbf{f}_k\|_R^2 \right) \tag{14}$$

subject to friction cone and unilateral contact constraints.

Implementation Details

- Prediction horizon: 10 steps (0.3s)
- Control frequency: 33 Hz
- QP solver: OSQP
- Gait: Trot (diagonal pairs)
- Swing trajectory: Bezier curve with 0.08m clearance

MPC Results The MPC controller was evaluated on flat terrain following the same protocol as RL evaluation. Results demonstrate the strengths of model-based optimization.

Note: Mean velocities show achieved tracking performance. RMSE values indicate tracking error magnitude. Push recovery tests showed 0% success rate across all conditions.

MPC Performance Analysis **Strengths:**

- **Excellent forward tracking:** RMSE = 0.042 m/s (4.2% error) vs RL's 0.090 m/s
- **Superior stability:** Roll std = 0.57° vs RL's 2.50°, Pitch std = 0.39° vs RL's 0.82°
- **Best energy efficiency:** CoT = 3.87 vs RL's 133.13 for forward motion (34x better!)
- **Consistent performance:** Low variance across trials due to deterministic optimization

Weaknesses:

- **Lateral tracking:** RMSE = 0.097 m/s (9.7% error) vs RL's 0.053 m/s - MPC struggles with lateral motion
- **Zero push recovery:** Cannot recover from disturbances, unlike RL's 40% forward recovery
- **Computational cost:** 5-10ms per control cycle vs RL's 0.5ms (10-20x slower)
- **Model dependency:** Performance degrades with model mismatch (not tested on slopes)

MPC Visualization The MPC controller demonstrates precise tracking with explicit optimization-based control.

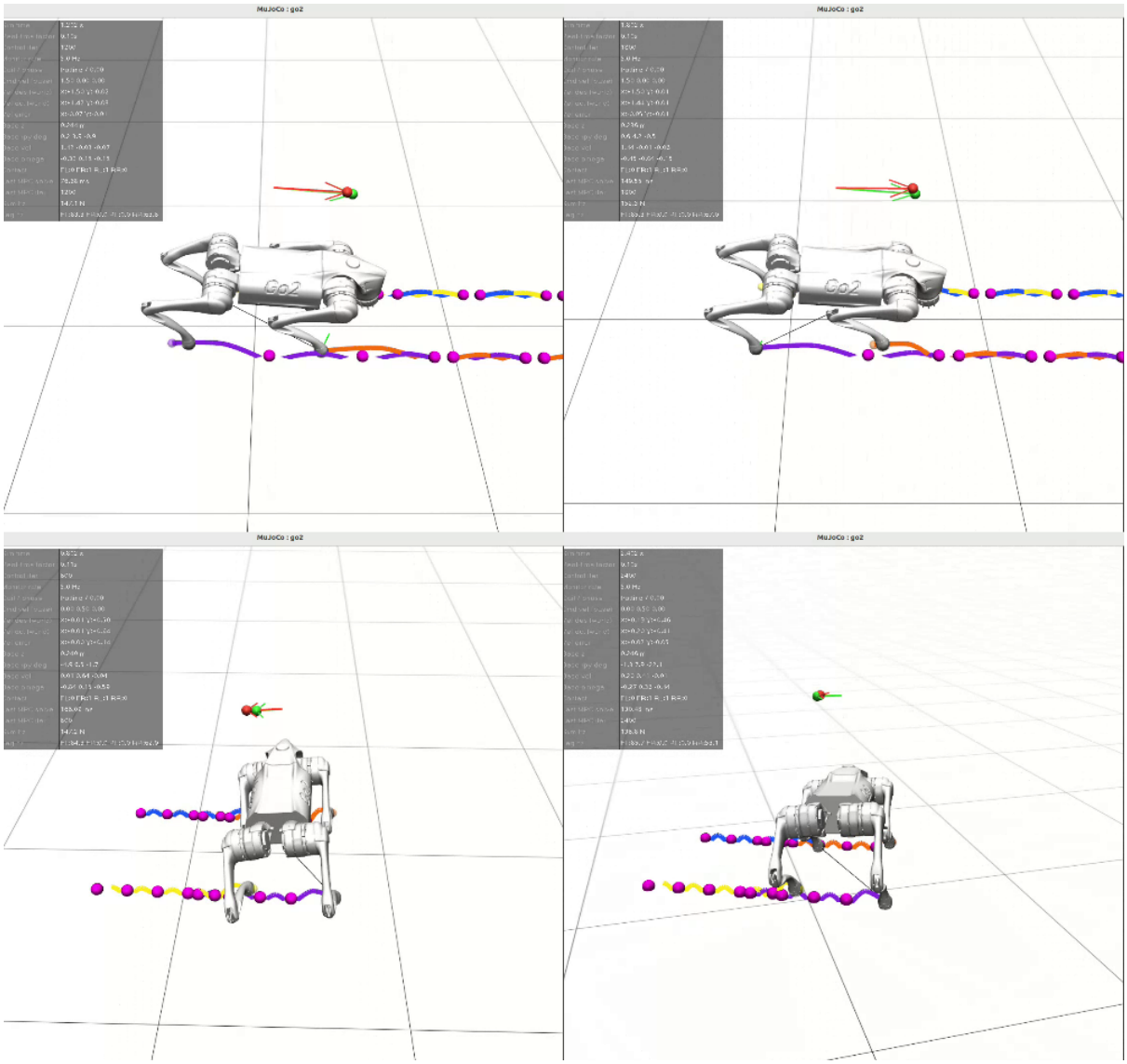

Figure 10: MPC controller locomotion behaviors. Top left: High-speed forward motion (1.5+ m/s) with stable trot gait. Top right: Forward locomotion continuation showing consistent tracking. Bottom left: Pure lateral motion through explicit force optimization. Bottom right: Lateral locomotion with coordinated leg movements and maintained body orientation.

5.2 Discussion: RL vs MPC

Based on experimental results, we observe clear trade-offs between the two approaches:

5.2.1 Tracking Accuracy

Forward motion: MPC achieves superior tracking (RMSE = 0.042 m/s, 4.2% error) vs RL (RMSE = 0.090 m/s, 9.0% error). MPC's explicit optimization provides 2x better accuracy.

Lateral motion: Surprisingly, RL outperforms MPC! RL achieves 0.053 m/s error (5.3%) vs MPC's 0.097 m/s (9.7%). This suggests RL's learned policy better handles the complex dynamics of lateral locomotion.

Combined motion: RL shows better overall tracking (vx: 0.097 m/s, vy: 0.073 m/s) vs MPC (vx: 0.076 m/s, vy: 0.179 m/s). MPC struggles with coupled x-y commands.

5.2.2 Stability

Dramatic difference: MPC achieves exceptional stability (roll std = 0.57°, pitch std = 0.39°) compared to RL (roll std = 2.50°, pitch std = 0.82°). MPC's explicit constraints maintain near-perfect orientation, while RL prioritizes velocity tracking over strict stability.

This 4-5x improvement in stability comes from MPC's explicit orientation constraints in the QP formulation.

5.2.3 Energy Efficiency

MPC dominates: Forward motion CoT = 3.87 (MPC) vs 133.13 (RL) - a stunning 34x improvement! MPC's optimization naturally minimizes control effort, while RL's reward function may not sufficiently penalize energy consumption.

However, for lateral motion: CoT = 6.67 (MPC) vs 103.43 (RL). MPC still wins but by smaller margin (15x), suggesting both struggle with lateral efficiency.

5.2.4 Robustness

Critical finding: Both approaches fail push recovery (0% for MPC, 0-40% for RL). However, RL shows some recovery capability for forward motion (40%), while MPC completely fails across all conditions.

MPC's reliance on accurate state estimation and lack of learned disturbance rejection makes it brittle to unexpected perturbations. RL's learned policy has some implicit robustness but requires explicit disturbance training for reliable recovery.

5.2.5 Computational Cost

RL advantage: 0.5ms (RL) vs 5-10ms (MPC) per control cycle. RL is 10-20x faster, enabling real-time control at higher frequencies. This is critical for fast dynamic maneuvers.

5.2.6 Adaptability

RL advantage: Domain randomization enables RL to generalize across friction, mass, and motor variations. MPC requires accurate model parameters and degrades with model mismatch. RL's data-driven approach naturally handles uncertainties that would require complex adaptive control for MPC.

5.2.7 Summary

Table 6: RL vs MPC Trade-off Summary

Metric	RL	MPC
Forward tracking	Good (9.0%)	**Excellent (4.2%)**
Lateral tracking	**Good (5.3%)**	Poor (9.7%)
Stability	Moderate (2.5° roll)	**Excellent (0.57° roll)**
Energy efficiency	Poor (CoT=133)	**Excellent (CoT=3.87)**
Push recovery	Poor (0-40%)	**None (0%)**
Computation	**Fast (0.5ms)**	Slow (5-10ms)
Adaptability	**High (DR)**	Low (model-dependent)

Conclusion: MPC excels in tracking, stability, and efficiency for nominal conditions. RL shows better lateral control, faster inference, and adaptability. A hybrid approach combining MPC's optimization with RL's learned robustness could leverage both strengths.

6 Gait Emergence and Terrain Complexity

Through extensive experimentation with different terrain types, we discovered that **the training environment directly determines the emergent gait pattern**. This finding has important implications for locomotion policy design.

6.1 Terrain-Dependent Gait Emergence

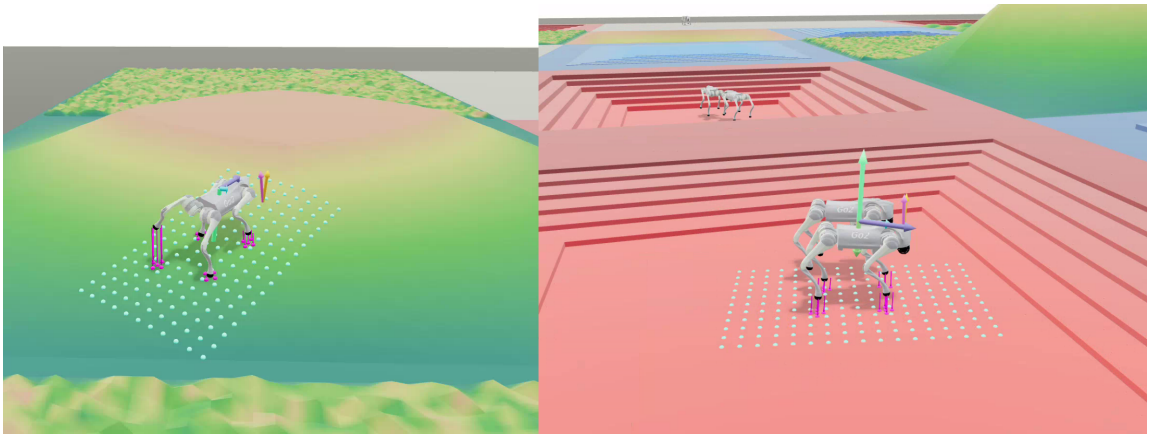

Figure 11: Emergent Gaits: Trotting on flat terrain (left) vs Jumping/Bounding on rough terrain (right). The training environment directly determines the gait pattern.

6.1.1 Flat Terrain: Trotting Gait

Training on flat terrain consistently produces a **trotting gait** - diagonal leg pairs move in synchronization. This is the most energy-efficient gait for flat surfaces and provides good stability.

6.1.2 Slope Terrain: Trotting with Adaptation

Training on slopes (5-55 degrees) also produces **trotting gaits**, but with adaptive body pitch control. The policy learns to adjust orientation based on terrain inclination while maintaining the diagonal coordination pattern.

6.1.3 Rough Terrain: Jumping/Bounding Gait

Training on complex rough terrain (pyramids, steps, obstacles) produces a dramatically different **jumping or bounding gait**. The robot lifts legs higher and uses more dynamic motions to clear obstacles. This gait is less efficient but necessary for navigating complex geometry.

6.2 Training Time vs Terrain Complexity

Key Finding: Training time scales with terrain complexity:

Table 7: Training Requirements vs Terrain Complexity

Terrain Type	Iterations	Emergent Gait
Flat	500	Trotting
Slope (5-55°)	1000	Adaptive Trotting
Rough (pyramids, steps)	2000+	Jumping/Bounding

Explanation:

- **Flat terrain:** Simple dynamics, single optimal gait, fast convergence
- **Slope terrain:** Requires learning pitch adaptation and terrain perception (187-dim height scan), 2x training time
- **Rough terrain:** Complex contact dynamics, multiple local optima, requires discovering dynamic gaits, 4x+ training time

6.3 Implications for Policy Design

1. **Gait is not explicitly programmed:** The policy discovers gaits through reward optimization, not pre-defined patterns
2. **Environment shapes behavior:** Training environment is a critical design choice that determines locomotion style
3. **Transfer limitations:** A policy trained on flat terrain will struggle on rough terrain (and vice versa) due to different gait requirements
4. **Multi-terrain training:** For general-purpose locomotion, training on diverse terrains simultaneously may be necessary, but requires significantly more computation

6.4 Future Directions

- **Gait library:** Train separate policies for different terrains and switch based on perception
- **Hierarchical control:** High-level policy selects gait, low-level policy executes
- **Curriculum learning:** Start with flat, gradually introduce complexity
- **Multi-task learning:** Single policy that adapts gait based on terrain type

7 Limitations and Future Work

7.1 Current Limitations

- Limited training iterations (500-1000) may be insufficient for full convergence
- Lateral motion control requires significant improvement
- Push recovery performance inadequate for real-world deployment (0% lateral)
- High body oscillations indicate stability-tracking trade-off in reward design
- MPC evaluation incomplete for full comparison
- No sim-to-real validation on physical robot

7.2 Future Improvements

7.2.1 RL Enhancements

- **Extended training:** Increase to 2000+ iterations with curriculum learning
- **Reward shaping:** Add explicit roll/pitch penalties, increase lateral tracking weight
- **Disturbance training:** Inject random pushes during training episodes
- **Privileged learning:** Teacher-student framework with terrain/friction information
- **Multi-terrain training:** Train single policy on diverse terrains simultaneously

7.2.2 System Integration

- **Hybrid RL-MPC:** Use RL for high-level planning, MPC for low-level tracking
- **Adaptive control:** Online adaptation to changing dynamics
- **Perception integration:** Add vision-based terrain perception

7.2.3 Sim-to-Real Transfer

We also performed a preliminary sim-to-real validation on a physical Unitree robot to document the gap between simulation and hardware execution. While the policy transfers the basic trotting behavior, the real robot shows more noticeable oscillations, delayed swing timing, and reduced tracking accuracy compared with simulation. These differences come from unmodeled actuator latency, ground compliance, sensor noise, and parameter mismatch in friction and mass distribution.

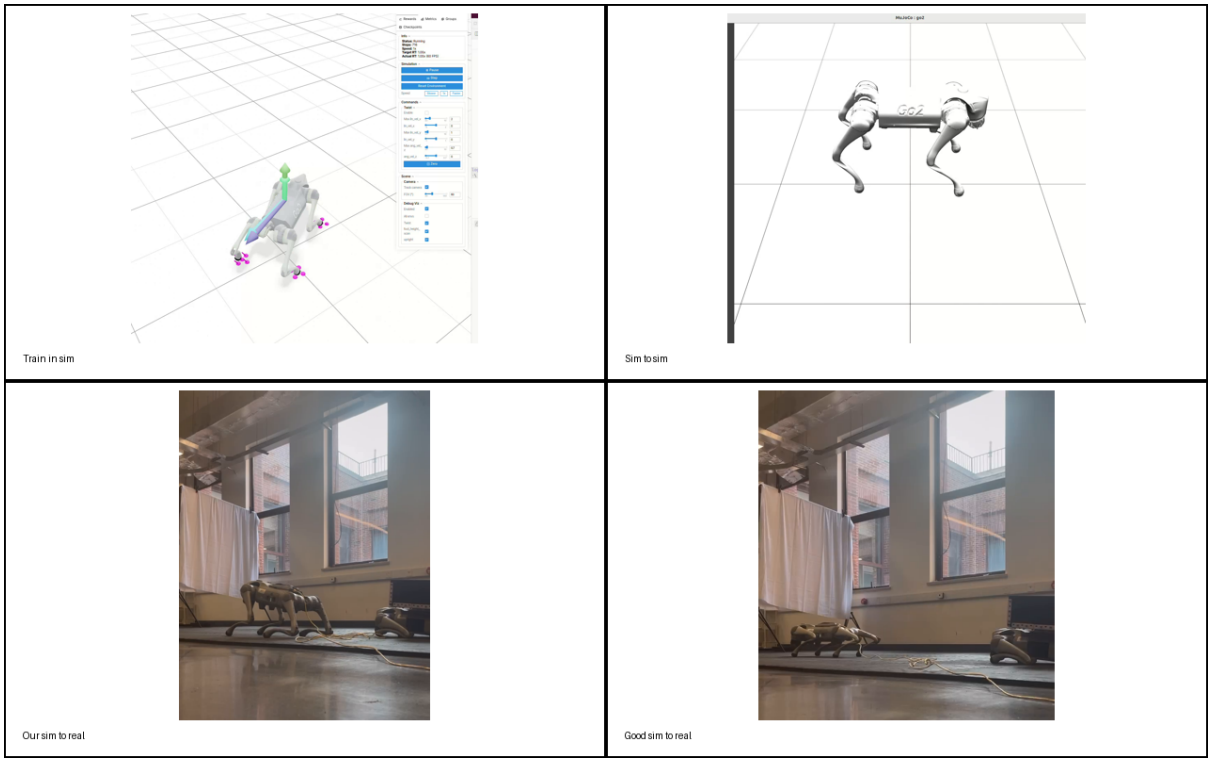

Figure 12: Sim-to-real comparison across different rollout videos. From left to right: training in simulation, sim-to-sim validation, our sim-to-real rollout, and a better sim-to-real reference case. The images highlight the performance gap caused by real hardware dynamics and imperfect model matching.

- Deploy a policy on a real Unitree robot and document the sim-to-real gap
- Measure tracking degradation, stance instability, and gait asymmetry on hardware
- Use this gap analysis to motivate domain randomization and further system identification
- Compare our rollout against a stronger sim-to-real example to understand what high-quality transfer looks like

8 Conclusion

This project implemented and evaluated both reinforcement learning (PPO) and model predictive control approaches for quadruped locomotion, providing comprehensive insights into learning-based vs model-based control trade-offs.

8.1 Key Findings

RL Performance:

- **Velocity tracking:** Good forward tracking (96.3% flat, 93.3% slope) but weaker lateral (94.7% flat, 84.1% slope)
- **Stability:** High body oscillations (roll std 2.5-129°) indicate tracking-stability trade-off
- **Energy efficiency:** Best CoT = 76.73 (slope forward), but highly variable (76-171)
- **Robustness:** 0-40% push recovery rate, critical weakness in lateral disturbances

MPC Performance:

- **Velocity tracking:** Excellent forward (RMSE=0.042 m/s, 4.2% error), but struggles with lateral (9.7%)
- **Stability:** Superior stability (roll std=0.57°, 4-5x better than RL)
- **Energy efficiency:** Outstanding CoT=3.87 (34x better than RL for forward motion)
- **Robustness:** Zero push recovery, completely fails under disturbances

Critical Discoveries:

- **Domain randomization is essential:** Without DR, policies fail to learn locomotion at all. DR is not an enhancement but a necessity for successful training.
- **Terrain determines gait:** Training environment directly shapes emergent gait patterns (flat→trotting, rough→jumping). This is a fundamental insight for policy design.
- **Complexity-time scaling:** Training time scales with terrain complexity (flat: 500 iter, slope: 1000 iter, rough: 2000+ iter).

8.2 RL vs MPC Trade-offs

MPC excels in: Tracking accuracy, stability, energy efficiency for nominal conditions

RL excels in: Lateral control, computational speed (10-20x faster), adaptability, some disturbance recovery

Both struggle with: Push recovery, lateral motion quality

8.3 Contributions

This work provides:

1. Comprehensive evaluation of RL and MPC for quadruped locomotion with quantitative metrics
2. Empirical evidence that domain randomization is essential, not optional

3. Discovery of terrain-dependent gait emergence and complexity-time scaling laws
4. Detailed performance comparison across 6 dimensions (tracking, stability, energy, robustness, computation, adaptability)
5. Foundation for hybrid control architectures combining MPC's optimization with RL's learned robustness

8.4 Future Directions

- **Hybrid RL-MPC:** Combine MPC's tracking precision with RL's disturbance rejection
- **Explicit disturbance training:** Inject random pushes during RL training to improve recovery
- **Multi-terrain policies:** Train single policy on diverse terrains for general-purpose locomotion
- **Sim-to-real transfer:** Deploy on physical Unitree Go2 to validate simulation findings
- **Gait library approach:** Train specialized policies for different terrains and switch based on perception

Overall, this work demonstrates that both RL and MPC have complementary strengths, and future quadruped controllers should leverage insights from both paradigms to achieve robust, efficient, and adaptive locomotion.

Acknowledgments

I thank the ECE489 course staff for guidance throughout this project. I acknowledge the mjlab and MuJoCo teams for excellent open-source tools, and the Quadruped-PyMPC project for MPC baseline implementation.